

Praktikum 10: Komplexität und Berechenbarkeit

Aufgabe 1: Cantorsches Diagonalargument

Angenommen, alle unendlichen Folgen über dem Alphabet $\{0, 1\}$ ließen sich in einer abzählbaren Liste aufzählen $\{f_1, f_2, f_3, \dots\}$, wobei $f_i(j)$ den j -ten Eintrag der i -ten Folge bezeichnet.

Betrachten Sie folgende (fiktive) Tabelle:

	1	2	3	4	5	...
Folge f_1 :	0	1	0	1	1	...
Folge f_2 :	1	0	1	0	0	...
Folge f_3 :	0	0	1	1	1	...
Folge f_4 :	1	1	0	0	1	...
Folge f_5 :	0	1	0	1	0	...

...

a) **Diagonalfolge konstruieren:**

Lesen Sie aus der obigen Tabelle die Diagonalelemente ab: $f_1(1), f_2(2), f_3(3), f_4(4), f_5(5)$.

Konstruieren Sie daraus eine neue Folge d , indem Sie jeden Diagonalwert negieren: $d(i) = 1 - f_i(i)$.

Geben Sie $d(1)$ bis $d(5)$ an.

b) **Widerspruch:**

Angenommen, d wäre die k -te Folge in der Liste, also $d = f_k$.

Zeigen Sie, dass $d(k) \neq f_k(k)$ gilt, und leiten Sie daraus einen Widerspruch ab.

c) **Schlussfolgerung:**

Begründen Sie: Warum gibt es mehr unendliche $\{0,1\}$ -Folgen als Python-Programme? Was folgt daraus für die Berechenbarkeit?

Aufgabe 2: Das Halteproblem

Angenommen, die folgende Funktion wäre berechenbar:

$$H(X, y) = \begin{cases} 0 & \text{falls Programm } X \text{ bei Eingabe } y \text{ terminiert} \\ 1 & \text{sonst} \end{cases}$$

Dann könnte man folgendes Programm $Z(y)$ schreiben:

$Z(y)$
1: repeat
2: until $H(Z, y) = 1$

a) **Widerspruchsbeweis:**

Zeigen Sie durch Fallunterscheidung, dass H nicht berechenbar sein kann:

b) **Konsequenz für die Praxis:**

In welchen realen Anwendungsfeldern spielt das Halteproblem eine Rolle? Nennen Sie zwei konkrete Beispiele und erklären Sie kurz, welche Einschränkung sich daraus ergibt.

Aufgabe 3: Komplexitätsklassen

a) **Einordnung:**

Ordnen Sie jedes der folgenden Probleme einer Klasse (P, NP, NP-vollständig oder nicht berechenbar) zu. Begründen Sie Ihre Entscheidung in einem Satz.

1. Gegeben n Zahlen – finde die größte.
2. Gegeben ein Programm X und Eingabe y – terminiert $X(y)$?
3. Gegeben eine boolesche Formel mit n Variablen – gibt es eine erfüllende Belegung? (Erfüllbarkeitsproblem / SAT)
4. Gegeben eine vorgeschlagene Belegung für SAT – ist sie korrekt?
5. Gegeben n Städte – was ist die kürzeste Rundreise? (Traveling Salesman Problem, TSP)
6. Gegeben eine vorgeschlagene TSP-Rundreise – ist ihre Länge $\leq k$?

b) **Ackermann-Funktion:**

Die Ackermann-Funktion ist definiert als:

$$\begin{aligned} A(0, n) &= n + 1 \\ A(m, 0) &= A(m - 1, 1) \\ A(m, n) &= A(m - 1, A(m, n - 1)) \end{aligned}$$

1. Berechnen Sie $A(1, 2)$ und $A(2, 2)$ schrittweise per Hand.
2. Implementieren Sie $A(m, n)$ rekursiv in Python.
3. Die Ackermann-Funktion ist berechenbar – gehört sie dennoch zu P? Begründen Sie.

Praktikumsbericht zu Versuch 10

Von Daniel Baer (3235621)

1. Cantorsches Diagonalargument

a) Diagonalfolge konstruieren

$$f_1(1) = 0$$

$$f_2(2) = 0$$

$$f_3(3) = 1$$

$$f_4(4) = 0$$

$$f_5(5) = 0$$

Daraus ergibt sich die Folge d : 1 1 0 1 1

b) Widerspruch

Wäre d die k -te Folge der Liste ($d = f_k$), gilt: $d(k) = f_k(k)$

Außerdem würde durch die Definition von d gelten: $d(k) = 1 - f_k(k)$

Daraus würde folgen: $f_k(k) = 1 - f_k(k)$

So entsteht der Widerspruch und d ist nicht die k -te Folge der Liste.

c) Schlussfolgerung

Jedes Python Programm besteht aus endlich vielen Befehlen und mehrere Programme können lexikographisch geordnet werden, sie sind abzählbar.

In unendlich binären Folgen kann sich immer ein Zeichen zu einer anderen Folge unterscheiden, die folgen sind nicht abzählbar oder durch ein Programm berechenbar.

2. Das Halteproblem

a) Widerspruchsbeweis

Falls $H(Z, y) = 0$ (1. Fall), terminiert das Programm nach Definition.

Das passiert nur wenn $H(Z, y) = 1$ (2. Fall), dann läuft das Programm allerdings nach Definition weiter.

So entsteht ein Widerspruch.

b) Konsequenz für die Praxis

Es gibt keine allgemeine Möglichkeit zu bestimmen, ob und wann ein Programm terminiert.

Ein Antivirus-Programm kann beispielsweise nicht nachvollziehen, ob ein Programm endlos weiterläuft.

Ebenfalls hat ein Compiler Probleme das Programm zu optimieren, wenn nicht erkannt wird, ob eine Funktion/Schleife beendet wird.

Diese Programme müssen mit Richtlinien und Beschränkungen arbeiten, um solche Fälle abzufangen.

3. Komplexitätsklassen

a) Einordnung

1. P, mit linearem Scan aller Zahlen.
2. nicht berechenbar, es kann nicht sicher bestimmt werden, ob ein Programm terminiert (Halteproblem).
3. NP-vollständig, es müssen im schlimmsten Fall alle Belegungen geprüft werden.
4. P, die Formel kann mit der Belegung ausgeführt werden.
5. NP-vollständig, es müssen im schlimmsten Fall alle Pfade berechnet werden.
6. P, die Länge kann polynomial berechnet werden.

b) Ackermann-Funktion

1. Berechnung per Hand

$$\begin{aligned} & A(1,2) \\ &= A(0, A(1,1)) \\ &= A(0, A(0, A(1,0))) \\ &= A(0, A(0, A(0,1))) \\ &= A(0, A(0,2)) \\ &= A(0,3) \\ &= 4 \end{aligned}$$

$$\begin{aligned} & A(2,2) \\ &= A(1, A(2,1)) \\ &= A(1, A(1, A(2,0))) \\ &= A(1, A(1, A(1,1))) && \text{(von oben } A(1,1) = 3) \\ &= A(1, A(1,3)) \\ &= A(1, A(0, A(1,2))) && \text{(von oben } A(1,2) = 4) \\ &= A(1, A(0,4)) \\ &= A(1,5) \\ &= A(0, A(1,4)) \\ &= A(0, A(0, A(1,3))) && \text{(bereits berechnet } A(1,3) = 5) \\ &= A(0, A(0,5)) \\ &= A(0,6) \\ &= 7 \end{aligned}$$

2. Implementierung

```
def ack(m: int, n: int) -> int:
    if m == 0:
        return n + 1
    if n == 0:
        return ack(m - 1, 1)
    return ack(m - 1, ack(m, n - 1))
```

3. Berechenbarkeit der Ackermann-Funktion

Die Ackermann-Funktion ist zwar berechenbar, aber nicht in polynomialer Zeit.

Die Berechnungsdauer Ackermann-Funktion wächst mit steigenden Argumenten extrem schnell.

So ist die Funktion nicht in P enthalten.